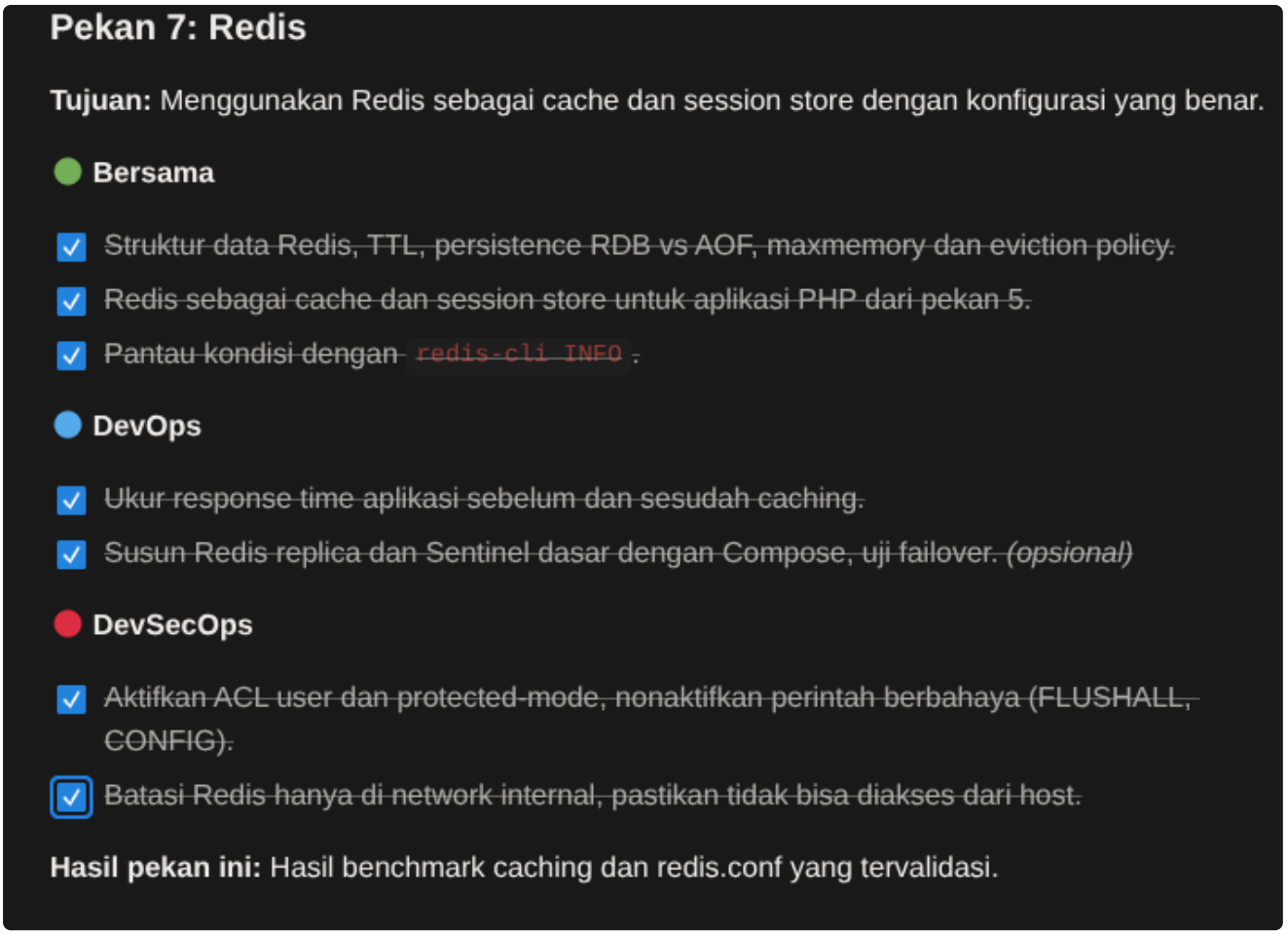
Dokumentasi Task 7

pada task 7 kali ini saya diberi beberapa task yang mana adalah



oke langsung aja pada point pertama yang mana struktur redis, ttl, persistence RDB vs AOF, max memory dan eviction policy

lalu kita akan langsung setup dulu redis nya agar penjelasannya lebih enak, langsung aja kita pakai command

docker volume create redis-data
docker run -d --name redis-latihan -p 6379:6379 -v redis-data:/data redis:7
docker exec -it redis-latihan redis-cli

nah setelah itu kira kira hasilnya kita akan masuk ke terminal redis nya yang mana hasil akhir nya kira kira seperti ini

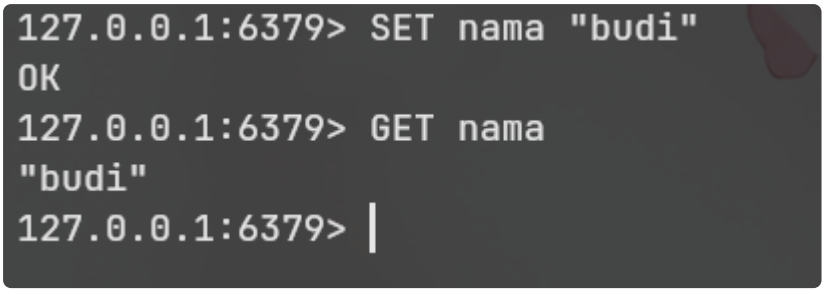


nah redis sendiri berfungsi sebagai storage data yang mana perbedaannya dengan dbms adalah redis ini menyimpan data yang banyak diakses oleh ram agar respon aplikasi ke hal hal kaya request get atau sejenisnya bisa berjalan sangat cepat dibanding kita tidak menggunakannya, nah langsung aja kita akan masuk ke struktur data pada redis dan disini redis punya 5 tipe utama data, yaitu:

a. string yang mana ini seperti string pada umumnya yang mana contoh perintahnya adalah

```
SET nama "budi"
GET nama
```

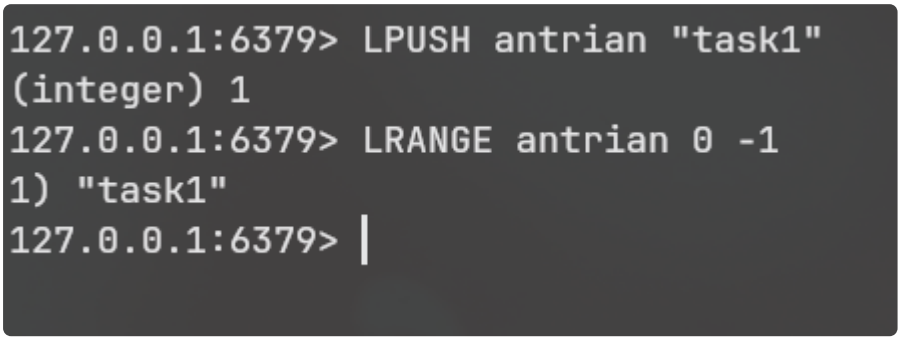
yang kira kira hasilnya adalah



b. list, nah ini mirip kaya array analoginya, dan contoh perintahnya adalah

```
LPUSH antrian "task1"
LRANGE antrian 0 -1
```

yang kira kira hasilnya sepert ini



c. set, nah ini kaya kumpulan nilai nilai yang unik gitu, dan ini tidak berurutan juga sih yang mana contoh perintahnya bisa dengan

```
SADD tags "redis" "cache"
SMEMBERS tags
```

yang kira kira hasilnya seperti ini

```
127.0.0.1:6379> SADD tags "redis" "cache"
(integer) 2
127.0.0.1:6379> SMEMBERS tags
1) "redis"
2) "cache"
127.0.0.1:6379>
```

d. hash, nah ini bukan hash yg di database tapi ini tuh kaya object dictionary, contohnya gini

```
HSET user:1 nama "budi"
HGETALL user:1
```

nah disitu kita juga bisa set nama umur jenis kelamin dll yang mana hasilnya kira kira kaya gini

```
127.0.0.1:6379> HSET user:1 nama "budi"
(integer) 1
127.0.0.1:6379> HSET user:1 jnskel "laki laki"
(integer) 1
127.0.0.1:6379> HGETALL user:1
1) "nama"
2) "budi"
3) "jnskel"
4) "laki laki"
127.0.0.1:6379>
```

e. sorted set, ini kaya set tapi tuh dia ada pengurutannya kaya sorting list digabung ama set gitu sihh, contoh command nya gini

```
ZADD leaderboard 100 "budi"
ZRANGE leaderboard 0 -1 WITHSCORES
```

yang mana hasilnya kira kira kaya gini

```
127.0.0.1:6379> ZADD leaderboard 100 "budi"
(integer) 1
127.0.0.1:6379> ZRANGE leaderboard 0 -1 WITHSCORES
1) "budi"
2) "100"
127.0.0.1:6379>
```

nah setelah itu kita akan lanjut ke TTL, ya ini time to live aja yang mana secara konsep mirip dengan ttl di jaringan biasa dan langsung aja kita akan mencoba untuk nge set ttl dengan comand

SET session:abc123 "data-user" EX 3600
TTL session:abc123

yang mana kira kira hasilnya seperti ini

```
127.0.0.1:6379> SET session:abc123 "data-user" EX 3600
OK
127.0.0.1:6379> TTL session:abc123
(integer) 3598
127.0.0.1:6379>
```

nah disitu EX 3600 itu berarti key tersebut akan expired setelah 3600 detik atau kira kira sejam, karena itu fitur ini sangat krusial untuk membuat sebuah session store yang mana sebuah session tidak perlu dihapus manual, tapi dia akan expired dengan sendirinya dan kita akan lanjut pada persistence RDB dan AOV dan kita langsung mulai dengan RDB yang mana RDB sendiri kurang lebih kaya snapshot dari tiap periode gitu dan AOV itu dia nyatet tiap log yang mengubah data yang ibaratnya kaya RDB itu journalctl dan AOV itu journald, dan langsung aja kita akan coba langsung dengan command

CONFIG SET appendonly yes
CONFIG SET save "60 100"

yang hasilnya kira kira kaya gini

```
127.0.0.1:6379> CONFIG SET appendonly yes
OK
127.0.0.1:6379> CONFIG SET save "60 100"
OK
127.0.0.1:6379>
```

nah bagian SET appendonly yes itu untuk mengaktifkan AOF dan SET sace "60 100" itu mengaktifkan RDB yang mana kira kira maksudnya adalah snapshot akan dibuat jika ada minimal 100 write dalam jangka waktu 60 detik. oke lanjut ke maxmemory dan eviction policy, maxmemory ada karena secara default redis itu tidak memiliki batas memori yang mana jika redis terus menggunakan ram maka server akan crash, nah untuk mengatasi itu ada yang namanya maxmemory dan kalau kita coba set ke 2gb dia akan menggunakan ram tersebut sampai 2 gb aja dan bagaimana jika ram penuh? ada yang namanya eviction policy yang mana simpelnya adalah avection policy itu kaya kita itu harus ngapain kalau ram penuh? nah kira kira penjelasannya dan avection policy itu ada berbagai jenis policy yang dipakai, kira kira seperti ini semua

Policy	**fungsi**
`noeviction`	nolak write baru kalau memori nya penuh

Policy	fungsi
`allkeys-lru`	intinya kita ngebuang key yang paling lama tidak diakses agar agak lega
`allkeys-lfu`	kalau ini kita ngebuang key yang paling jarang diakses agar agak lega juga
`volatile-lru`	sama kaya `allkeys-lru`, cuma untuk key yang punya TTL aja
`volatile-ttl`	buang key yang TTL nya bakalan habis

terus kita akan coba praktek dengan command dibawah ini:

CONFIG SET maxmemory 100mb
CONFIG SET maxmemory-policy allkeys-lru

yang hasilnya kira kira seperti ini

```
127.0.0.1:6379> CONFIG SET maxmemory 100mb
OK
127.0.0.1:6379> CONFIG SET maxmemory-policy allkeys-lru
OK
127.0.0.1:6379>
```

nah maksud dari command nya adalah kita membatasi max memory di 100mb dan kita juga membuat policy allkeys-lru yang artinya kita akan ngebuang key yang paling lama ga diakses lalu kita juga bisa verifikasi dengan command

redis-cli INFO memory

terus kira kira akan menghasilkan output:



nah disitu ada keterangan seperti

maxmemory:104857600
maxmemory_human:100.00M
maxmemory_policy:allkeys-lru

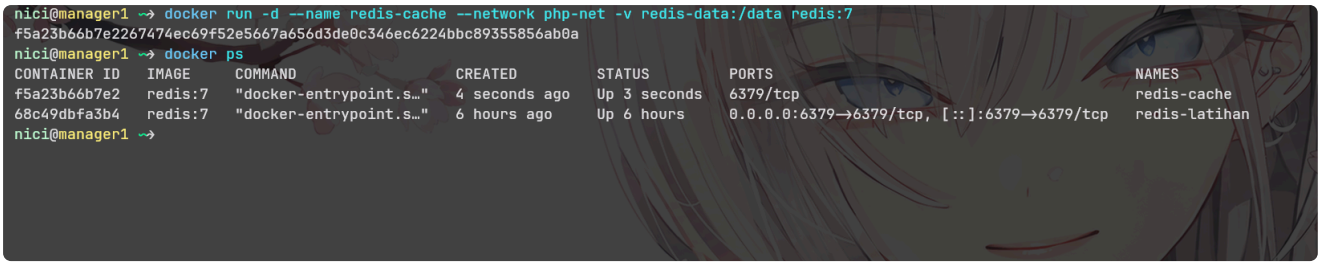
yang mana menandakan bahwa konfigurasi maupun policy yang kita apply itu jalan dengan baik. gitu sih dan lanjut ke poin berikutnya, yaitu redis sebagai session store untuk aplikasi php pekan 5, oke kalau kita callback pada pekan 5, kita sempat membuat sebuah aplikasi php sederhana yang pake php-fpm itu nah dan untuk alur tanpa redis itu kira kira seperti ini, kita konek terus nanti akan kirim query yang mana akan dieksekusi oleh mysql dan akan mengirim output kembali, nah kalau data yang jarang berubah tapi sering banyak yang request kaya marketplace, siklus itu akan berubah ubah mulu yang mana itu pemborosan

aja lalu kita akan mulai menggunakan redis agar data yang jarang berubah tapi kita sering request itu kita letakkan aja pada redis dan berikut step by stepnya:

pertama kita akan membuat container nya dulu dengan command

docker run -d --name redis-cache --network php-net -v redis-data:/data redis:7

lalu hasilnya akan seperti ini



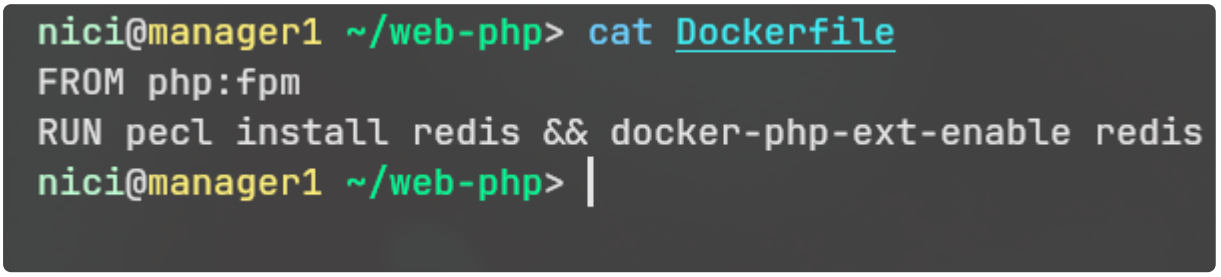
nah setelah itu lanjut untuk kita akan menambahkan sesuatu pada dockerfile nya untuk menginstall ekstensi redis pada php nya dengan command:

nano Dockerfile

lalu tambahkan dengan

FROM php:fpm
RUN pecl install redis && docker-php-ext-enable redis

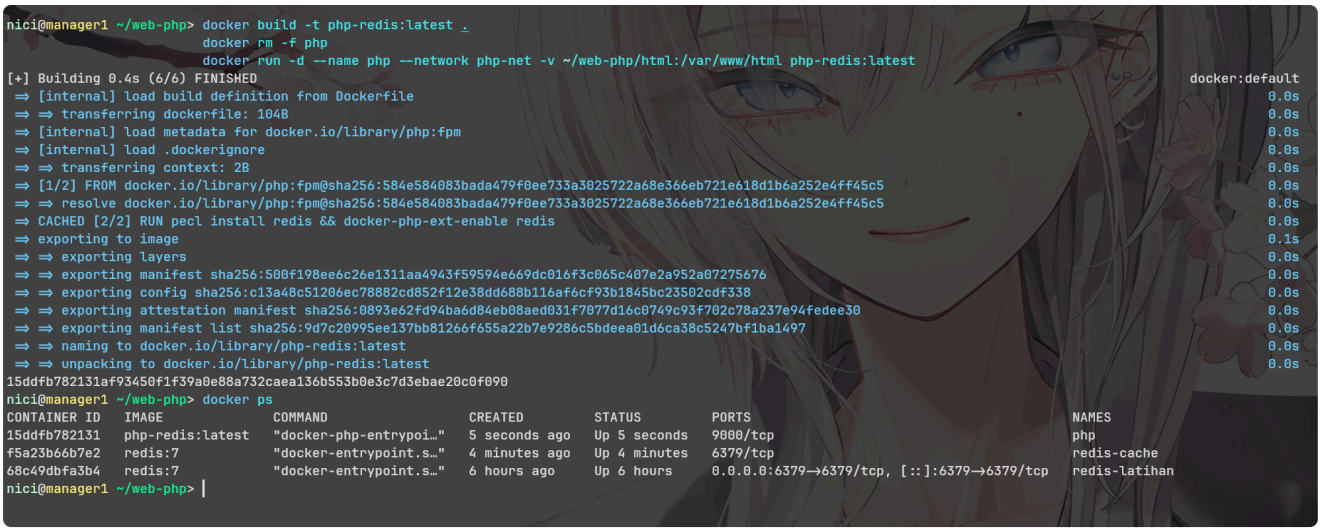
kira kira hasilnya akan seperti berikut:



nah pecl itu sebuah paket manajer khusus buat php, yaa kaya pacman, apt dll gitu kurleb dan docker-php-enable-ext itu untuk nge enable extension yang baru kita install, terus kita tinggal build aja dengan command

docker build -t php-redis:latest .
docker rm -f php
docker run -d --name php --network php-net -v ~/web-php/html:/var/www/html php-redis:latest

setelah itu hasilnya kira kira akan seperti ini



nah lalu aku menyuruh ai generate source code buat test cache aja yang kira kira berisi

```
<?php
```

$redis = new Redis(redis->connect('redis-cache', 6379);

$data = 'dt... h';cached = $redis->get(key);

if (
cached) { echo "Dari CACHE: " . $cached; } else { sleep(2); // simulasi query database yang lambat $data =
key, 60, $data); // simpan 60 detik
echo "Dari DATABASE (lambat): " . $data;
}

kira kira kaya gini

```
nici@manager1 ~/web-php> cat html/cache-test.php
<?php
$redis = new Redis();
$redis->connect('redis-cache', 6379);

$key = 'data-mahal';
$cached = $redis->get($key);

if ($cached) {
    echo "Dari CACHE: " . $cached;
} else {
    sleep(2); // simulasi query database yang lambat
    $data = "Hasil query jam " . date('H:i:s');
    $redis->setex($key, 60, $data); // simpan 60 detik
    echo "Dari DATABASE (lambat): " . $data;
}
nici@manager1 ~/web-php>
```

setelah itu langsung aja kita akan coba untuk tes aja dengan command curl yang mana command nya adalah

curl -s localhost:8080/cache-test.php
curl -s localhost:8080/cache-test.php

lalu hasilnya kira kira seperti ini

```
nici@manager1 ~/web-php> curl -s localhost:8080/cache-test.php
                         curl -s localhost:8080/cache-test.php
nici@manager1 ~/web-php [7]>
```

maaf itu gagal dikarenakan aku lupa menyalakan container nginx nya jadi kita akan nyalakan kembali dengan command

docker run -d --name web-nginx --network php-net -p 8080:80
-v ~/web-php/html:/var/www/html
-v ~/web-php/nginx.conf:/etc/nginx/conf.d/default.conf
nginx

lalu langsung aja tes ulang dengan command yang sama akan menghasilkan output seperti ini

```
Dari DATABASE (lambat): Hasil query jam 09:09:14Dari CACHE: Hasil query jam 09:09:14⏎
nici@manager1 ~/web-php> curl -s localhost:8080/cache-test.php
                         curl -s localhost:8080/cache-test.php
Dari CACHE: Hasil query jam 09:09:14Dari CACHE: Hasil query jam 09:09:14⏎
nici@manager1 ~/web-php>
```

Request pertama menghasilkan output "Dari DATABASE (lambat)" dengan delay kurleb 2 detik, lalu request kedua (dalam 60 detik) menampilkan "Dari CACHE" secara instan yang membuktikan caching bekerja.

oke lalu kita akan lanjut dengan session store via redis, ya kira kira untuk cara kerja nya udah paham lah ya jadi kita akan langsung akan fokus ke command command nya

jadi kita akan mengubah session store nya ke redis di dalam file php.ini dan jika belum ada bisa dibuat dengan command

nano php.ini

atau

touch php.ini

lalu isi dengan

session.save_handler = redis
session.save_path = "tcp://redis-cache:6379"

nah kita mengubah sesi nya ke redis dan menambahkan path nya ke path redis dan coba langsung aja kita hapus container yang udah nyala dan membuat container baru dengan command

docker rm -f php
docker run -d --name php --network php-net
-v ~/web-php/html:/var/www/html
-v ~/web-php/php.ini:/usr/local/etc/php/conf.d/redis-session.ini
php-redis:latest

dan setelah itu kita jangan lupa membuat kode php untuk session nya dan kalau file nya belum ada kita bisa buat aja langsung dengan command

nano /html/session-test.php

dan isi dengan

```
<?php
```

session_start();
$_SESSION['hits'] = (_SESSION['hits'] ?? 0) + 1;
echo "Jumlah kunjungan: " . $_SESSION['hits'];

lalu kita akan langsung tes aja ke url nya yang mana kita bisa menggunakan command

curl -c cookie.txt -b cookie.txt localhost:8080/session-test.php

dan akan menghasilkan output

```
php
2ff7520a64a631e40b107dfa5d653778fdfa359885f058aaa22df97b9b17476e
nici@manager1 ~/web-php> curl -c cookie.txt -b cookie.txt localhost:8080/session-test.php
Jumlah kunjungan: 1
nici@manager1 ~/web-php>
```

dan jika kita spam command nya maka jumlah kunjungan akan bertambah kaya gini

```
nici@manager1 ~/web-php> curl -c cookie.txt -b cookie.txt localhost:8080/session-test.php
Jumlah kunjungan: 1
nici@manager1 ~/web-php> curl -c cookie.txt -b cookie.txt localhost:8080/session-test.php
Jumlah kunjungan: 2
nici@manager1 ~/web-php> curl -c cookie.txt -b cookie.txt localhost:8080/session-test.php
Jumlah kunjungan: 3
nici@manager1 ~/web-php> curl -c cookie.txt -b cookie.txt localhost:8080/session-test.php
Jumlah kunjungan: 4
nici@manager1 ~/web-php> curl -c cookie.txt -b cookie.txt localhost:8080/session-test.php
Jumlah kunjungan: 5
nici@manager1 ~/web-php> curl -c cookie.txt -b cookie.txt localhost:8080/session-test.php
Jumlah kunjungan: 6
nici@manager1 ~/web-php> curl -c cookie.txt -b cookie.txt localhost:8080/session-test.php
Jumlah kunjungan: 7
nici@manager1 ~/web-php>
```

nah untuk session seperti itu dan kita akan lanjut ke mantau kondisi redis dengan menggunakan command redis-cli INFO yang mana untuk command command nya itu ada

docker exec -it redis-latihan redis-cli INFO

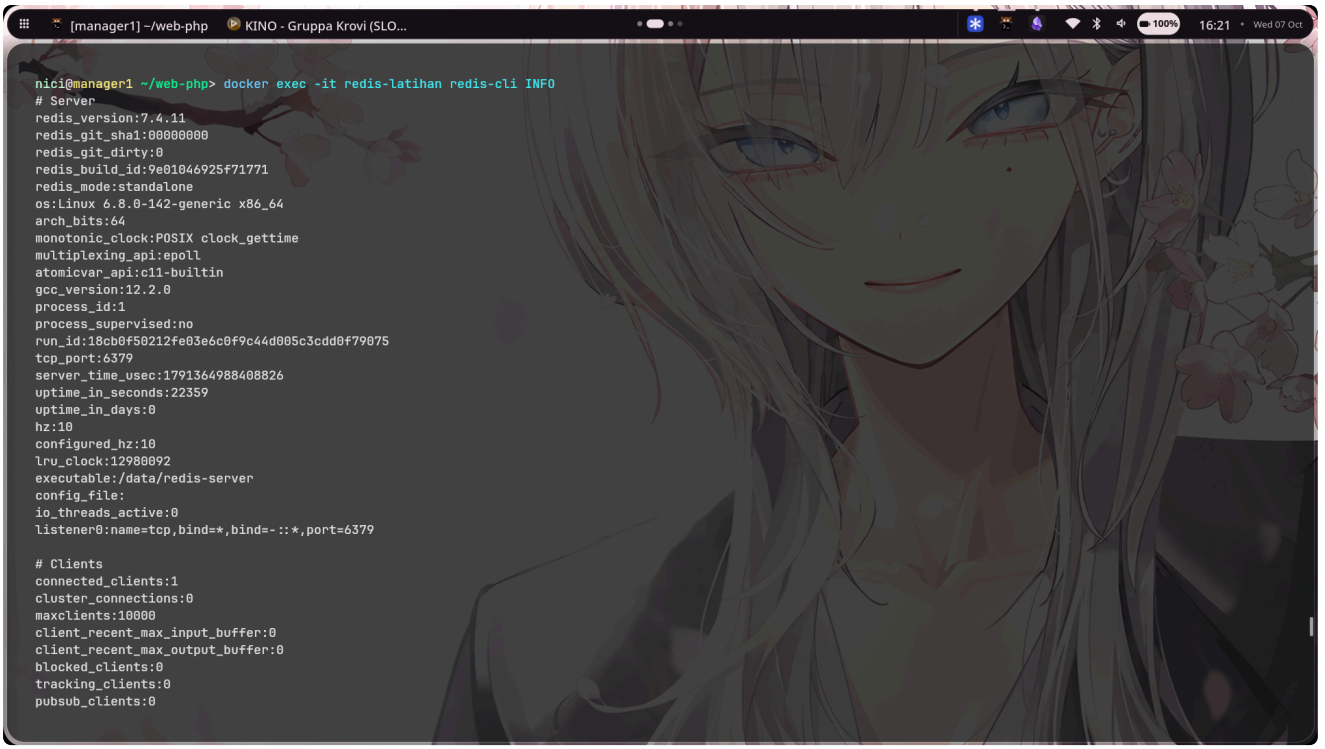
itu untuk general, lalu untuk memori ada:

docker exec -it redis-latihan redis-cli INFO memory

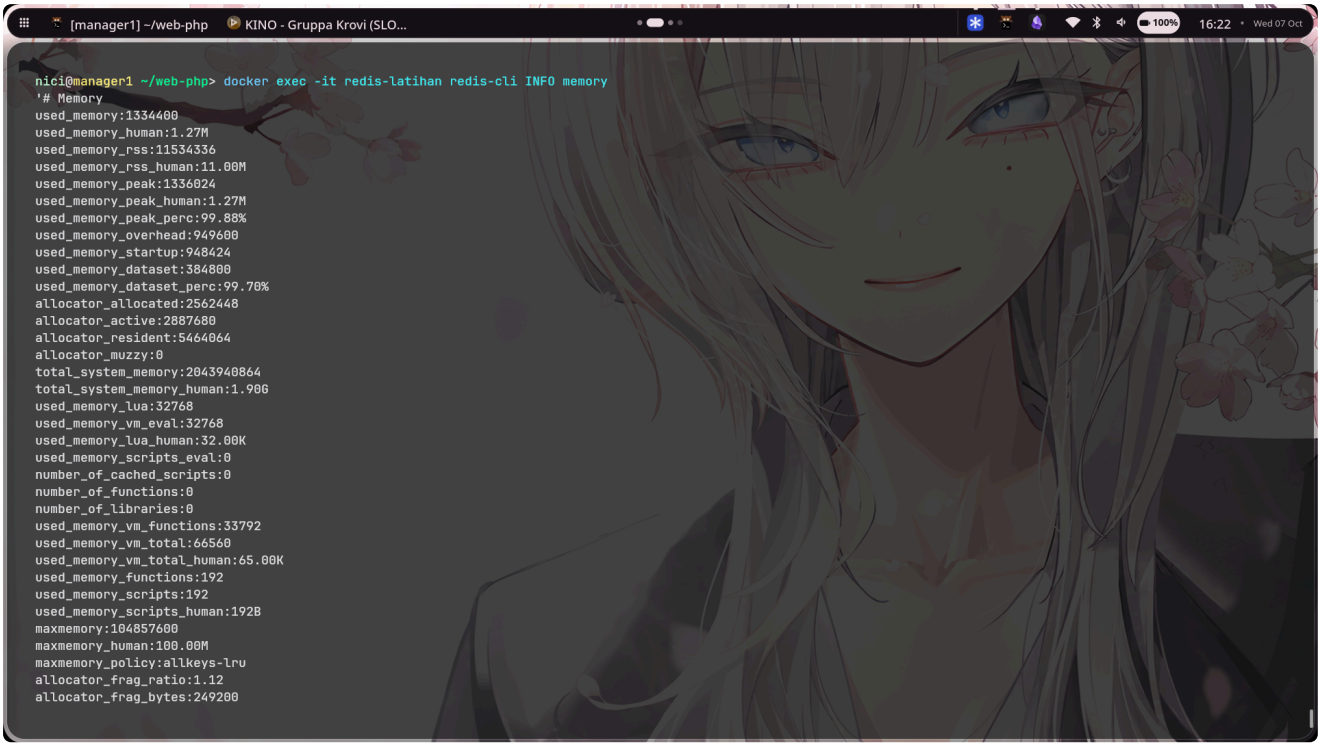
lalu statistik menggunakan command

docker exec -it redis-latihan redis-cli INFO stats

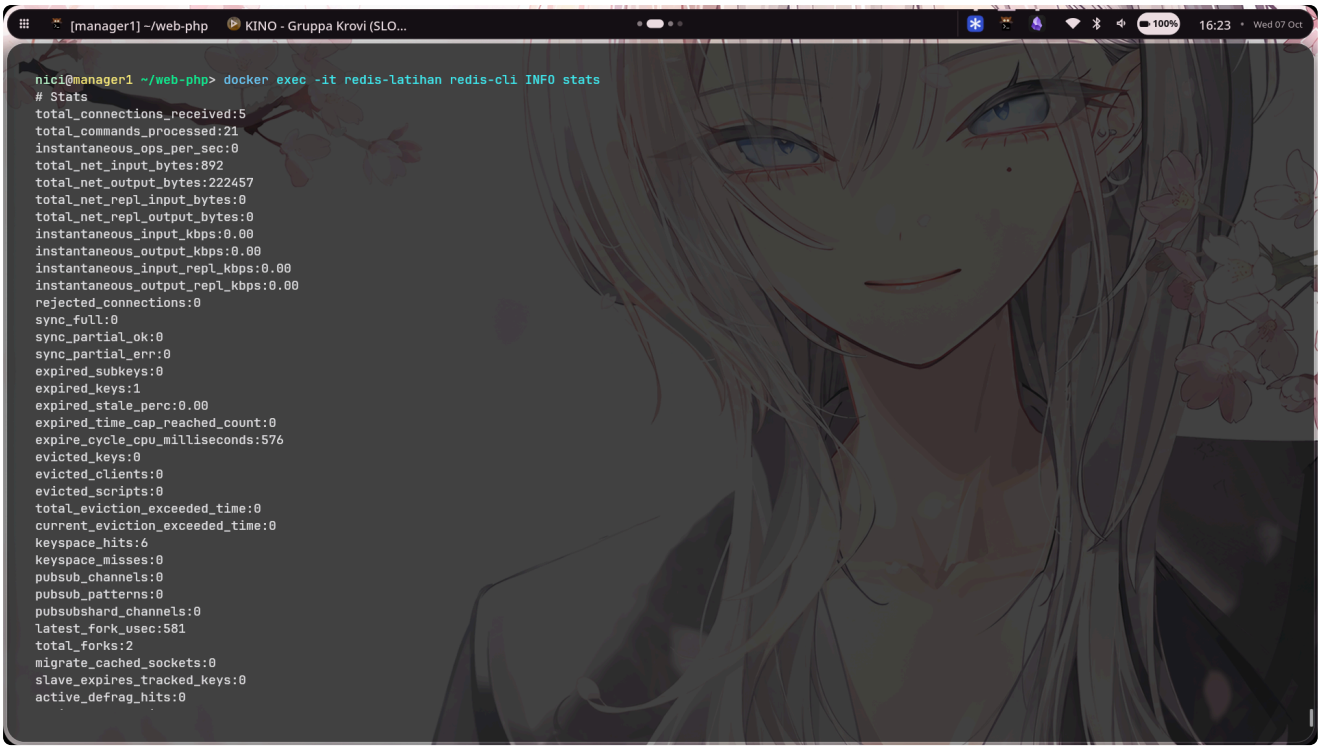
yang mana akan kita lakukan satu satu yang mana jika berurutan akan menghasilkan output:



itu se abrek dan lanjut ke memori



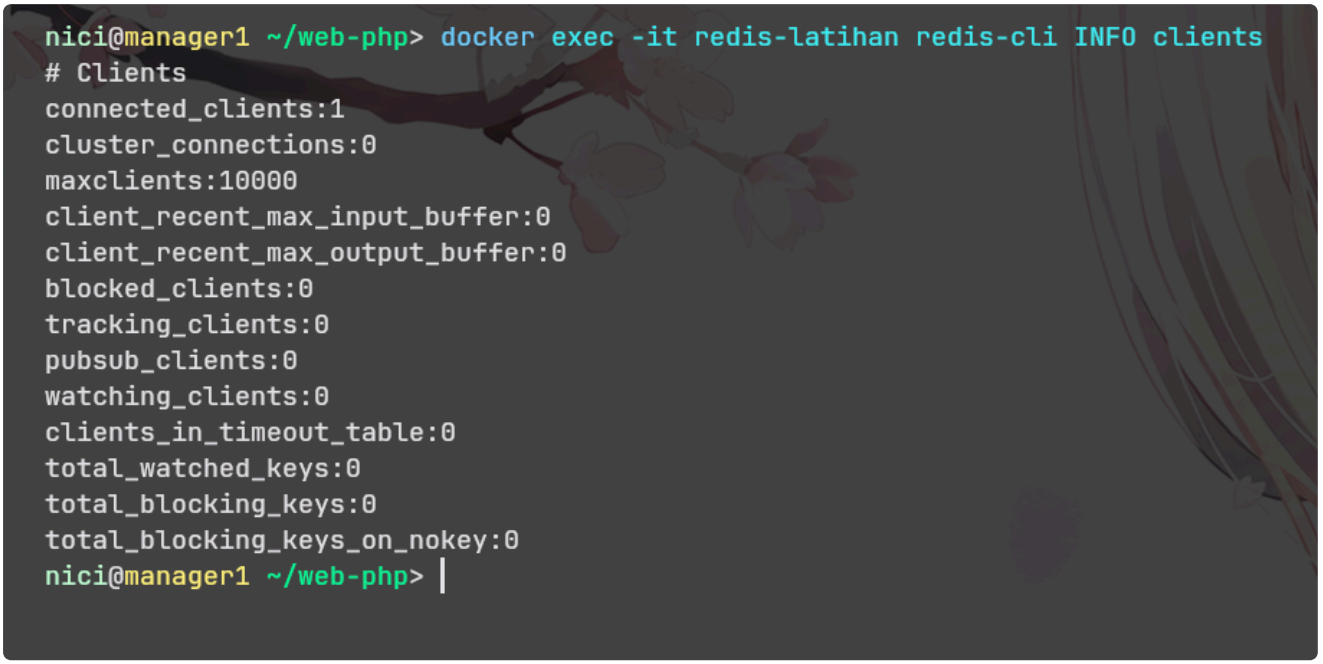
seabrek juga dan statistik



oh ya kalau mau cek klien nya bisa dengan command

docker exec -it redis-latihan redis-cli INFO clients

yang hasilnya kaya gini



lalu kita membuat sebuah redis replica dan sentinel dengan compose, yang mana replica ini kurang lebih itu kurang lebih konfigurasinya terdiri dari sebuah redis master dan ada beberapa replica yang selalu menerima data dari master dan jika master mati maka data akan tetap ada pada replica, lalu sentinel kurang lebih itu konfigurasinya terdiri dari sebuah sentinel yang mana dia memonitor master dan replica yang lain, dan jika ada salah satu master yang mati maka sentinel akan mempromosikan replica lain sebagai master, kurang lebih mirip dengan cara kerja DR atau BDR di ospf tapi hanya secara garis besar saja, oke langsung aja kita mulai konfigurasi step by stepnya adalah

kita akan membuat sebuah direktori dengan command

mkdir -p redis-ha

lalu di dalam redis-ha kita membuat sebuah compose dengan command

nano compose.yaml

isi dengan

```
services:
redis-master:
image: redis:7
command: redis-server --appendonly yes

redis-replica:
image: redis:7
command: redis-server --replicaof redis-master 6379

sentinel:
image: redis:7
command: redis-sentinel /etc/redis/sentinel.conf
volumes:
- ./sentinel.conf:/etc/redis/sentinel.conf
depends_on:
- redis-master
- redis-replica
restart: on-failure
```

lalu langsung aja kita nyalakan dengan command

docker compose up -d

yang hasilnya kira kira seperti ini

```
nici@manager1 ~/redis-ha> docker compose up -d
[+] up 3/3
 ✓ Container redis-ha-redis-master-1  Started
 ✓ Container redis-ha-redis-replica-1 Started
 ✓ Container redis-ha-sentinel-1      Started
nici@manager1 ~/redis-ha>
```

nah setelah itu kita akan coba uji failover dengan command berikut

docker exec -it redis-ha-redis-master-1 redis-cli SET testkey "sebelum-mati"
docker stop redis-ha-redis-master-1
sleep 15
docker exec -it redis-ha-sentinel-1 redis-cli -p 26379 SENTINEL get-master-addr-by-name mymaster

nah setelah itu kira kira akan menghasilkan output

```
nici@manager1 ~/redis-ha> docker compose up -d
[+] up 3/3
 ✓ Container redis-ha-redis-master-1  Started
 ✓ Container redis-ha-redis-replica-1 Started
 ✓ Container redis-ha-sentinel-1      Started
nici@manager1 ~/redis-ha> docker exec -it redis-ha-redis-master-1 redis-cli SET testkey "sebelum-mati"
                          docker stop redis-ha-redis-master-1
                          sleep 15
                          docker exec -it redis-ha-sentinel-1 redis-cli -p 26379 SENTINEL get-master-addr-by-name mymaster

OK
redis-ha-redis-master-1
1) "172.23.0.2"
2) "6379"
nici@manager1 ~/redis-ha>
```

lalu kita juga bisa konfirmasi ulang apakah ip yang keluar itu punya nya redis replica atau emang itu sebenernya hasil dari master aja yang masih nyangkut, dan saat aku mencoba itu menghasilkan output

```
nici@manager1 ~/redis-ha> docker inspect redis-ha-redis-replica-1 --format '{{(index .NetworkSettings.Networks "redis-ha_default").IPAddress}}'
172.23.0.2
```

yang mana dari output itu kita bisa tau kalau ip itu benar benar milik replica

oke next kita akan coba security pada redis nya, yang mana redis itu menggunakan protected mode untuk membatasi koneksi dari luar localhost jika autentikasi belum dikonfigurasi. cuma kalau setelah itu kita mengaktifkan autentikasi autentikasi, akses dari luar dapat digunakan sesuai konfigurasi. nah lalu ada yang namanya ACL, ya ACL ini basic

lah ya cuma langsung kalo mau konfigurasi bisa langsung aja masuk ke redis nya dengan command

docker exec -it redis-ha-redis-replica-1 redis-cli

lalu di dalam redis ketik command

ACL SETUSER app_user on >app_pass123 ~cache:* +get +set +setex -flushall -config

lalu hasilnya harusnya ok semua kaya gini

```
127.0.0.1:6379> ACL SETUSER app_user on >app_pass123 ~cache:* +get +set +setex -flushall -config
OK
```

lalu kita akan melakukan tes acl nya dengan beberapa command di dalam redisnya, command nya sebagai berikut

AUTH app_user app_pass123
SET cache:test "data"
GET cache:test
FLUSHALL

yang mana kalau kita jalankan akan menghasilkan error pada flushall karena app_pass123 tidak diberi permission untuk melakukan itu, kira kira hasilnya gini

```
127.0.0.1:6379> ACL SETUSER app_user on >app_pass123 ~cache:* +get +set +setex -flushall -config
OK
127.0.0.1:6379> AUTH app_user app_pass123
OK
127.0.0.1:6379> SET cache:test "data"
OK
127.0.0.1:6379> GET cache:test
"data"
127.0.0.1:6379> FLUSHALL
(error) NOPERM User app_user has no permissions to run the 'flushall' command
127.0.0.1:6379> exit
```

disitu command FLUSHALL itu ditolak yang mana itu membuktikan bahwa ACL nya bekerja dengan baik, next kita akan disable command command suspicious dengan global , langsung aja kita bikin sebuah file yang kita namai dengan redis.conf dengan command

nano redis.conf

lalu isi dengan ini

protected-mode yes

user default on >redis_root_pass123 ~ & +@all

rename-command FLUSHALL ""
rename-command FLUSHDB ""
rename-command CONFIG ""

```
nici@manager1 ~/redis-ha> cat redis.conf
protected-mode yes

user default on >redis_root_pass123 ~* &* +@all

rename-command FLUSHALL ""
rename-command FLUSHDB ""
rename-command CONFIG ""
nici@manager1 ~/redis-ha>
```

anw command rename-command itu kira kira kita mengganti command yang dituju menjadi string kosong kalau pada konfigurasinya, kalau diganti ke yang lain juga bisa, ibarat sensor pada film yang mana aslinya aktor bilang anjing tapi karena dinilai kasar jadi diganti kucing yang mana sama kaya itu, command berbahaya kaya FLUSHALL, FLUSHDB dll itu akan diganti ke string kosong jadi kalaupun itu dijebol ya attacker susah kalau mau akses command command tersebut karena apa yang mereka ketik diganti menjadi string kosong yang mana tidak ada command apapun yang ter eksekusi, coba kita langsung aja ter kriuk dengan membuat container baru dan menggunakan redis.conf itu sebagai satpam nya jadi langsung aja kita eksekusi dengan command

docker run -d --name redis-secured -v ~/redis.conf:/usr/local/etc/redis/redis.conf -v redis-data:/data redis:7 redis-server /usr/local/etc/redis/redis.conf

yang mana outputnya harusnya gini

```
nici@manager1 ~/redis-ha> docker run -d --name redis-secured -v ~/redis.conf:/usr/local/etc/redis/redis.conf -v redis-data:/data redis:7 redis-server /usr/local/etc/redis/redis.conf
b2853ad79966254f4f31a471425c81b74478120639d561161ce25345a2a28ed0
nici@manager1 ~/redis-ha>
```

mari tes kriuk dengan mencoba dua command ini

docker exec -it redis-secured redis-cli -a redis_root_pass123 ACL WHOAMI
docker exec -it redis-secured redis-cli -a redis_root_pass123 FLUSHALL

lalu hasilnya akan default ama blong ama error aja saja seperti ini

```
nici@manager1 ~/redis-ha> docker exec -it redis-secured redis-cli -a redis_root_pass123 ACL WHOAMI
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
```

```
nici@manager1 ~/redis-ha> docker exec -it redis-secured redis-cli -a redis_root_pass123 FLUSHALL
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
(error) ERR unknown command 'FLUSHALL', with args beginning with:
nici@manager1 ~/redis-ha>
```

, dan yang terakhir ada membuat redis ini berjalan hanya pada di network internal, ya ini gampang sih, kita compose down dulu dengan command

docker compose down

lalu kita ketik

nano compose.yaml

lalu isi dengan

services:
redis-master:
image: redis:7
command: redis-server --appendonly yes

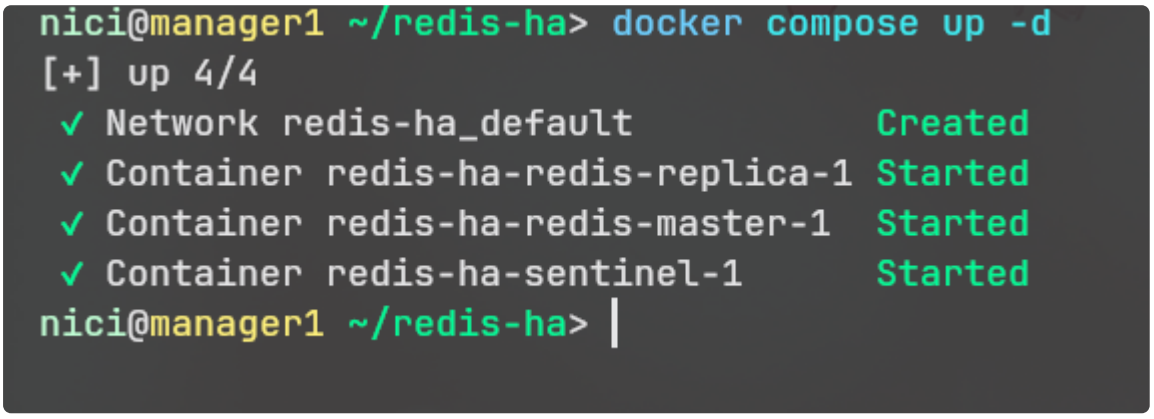
redis-replica:
image: redis:7
command: redis-server --replicaof redis-master 6379

sentinel:
image: redis:7
expose:
- "6379"
command: redis-sentinel /etc/redis/sentinel.conf
volumes:
- ./sentinel.conf:/etc/redis/sentinel.conf
depends_on:
- redis-master
- redis-replica
restart: on-failure

bagian mana yang mengandung konfigurasi agar redis hanya konek ke jaringan internal aja?
bagian expose, nah tinggal compose up lagi deh

docker compose up -d

hasilnya gini



udah gitu aja, kita akan coba tes koneksi dari host ke redis apakah cinta kita diterima atau di tolak dengan command

redis-cli -h 127.0.0.1 -p 6379 PING



nah itu udah ketolak, sekian dokumentasi task 7 ini, terimakasih